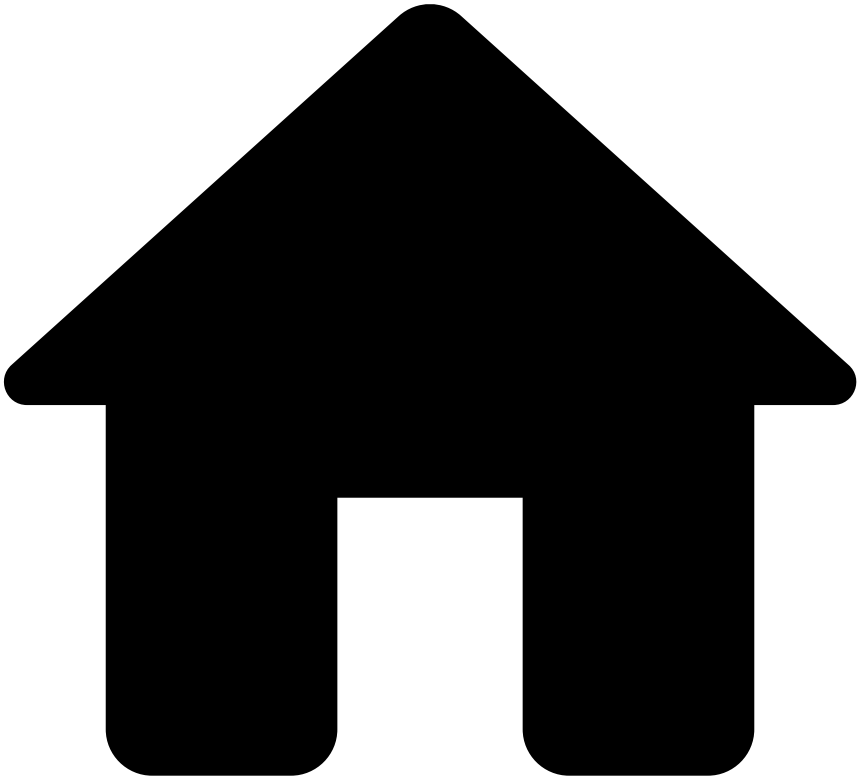


Case Manager Event Ingestion

Case Manager can ingest different types of events:

- Events (i.e. transactions)
- Status Changes
- Related Transactions

This page focuses on the first type: events, which are the most prevalent.

Overviewâ□□

Events are, essentially, business transactions. Multiple events, however, can relate to the same transaction. For example, a transaction may have a first event containing information about a purchase that was made by a customer. Later, the

customer may change the shipping address, which will cause a second (update) event to be sent to Case Manager, notifying it of the address change. Internally, CM will perform the necessary actions to update the shipping address of the relevant transaction.

With this in mind, Case Manager can receive two types of events:

- **New events** (the first of a certain transaction)
- **Update events** (an update to a transaction CM has already seen)

Bear in mind that these types of events may arrive out of order: ahead we will explain how Case Manager handles this situation. This means that any event of a particular transaction that arrives **first** willÂ **always** be considered as aÂ **new event**, regardless of whether it's an update or the original. This is because CM hasn't seen that transaction before, so it must assume it's new. So, if Case Manager receives an update event but it has never seen that transaction before, it assumes that the event is new and deals with it as such. Later, when the actualÂ **original** event arrives, CM is intelligent enough to perform the necessary actions so that the final result, in the database, is the same as if the events had arrived in their original order.

For update events, there is also the notion of aÂ **full** vs.Â **partial** update. A full update simply means that all of the fields that are present in theÂ update event will be used to update the current fields present in Case Manager. A partial update, however, will only update a configurable subset of the fields, as we'll see ahead. Besides this, there's another difference between the two: partial updates explicitly state, through one of their fields, that they are in fact an update. By configuring this, Case Manager will then be able to tell, through the event payload, if it's dealing with a partial update or not. This contrasts with "regular" updates, which areÂ **inferred** by Case Manager merely by checking if it has already seen that transaction or not, as we saw before. Here's a simple diagram denoting the difference:

Event Ingestion Flowâ□□

Now that we know the difference between new events and updates, let's take a look at the main event ingestion flow. In order to be fully stored in CM's database, an event goes through a series of steps which are what weÂ call CM's event ingestion.

Determine whether the event is new or an updateâ□□

The first step in the event ingestion flow determines if the current event is new (i.e. the first of a transaction) or an update. In order to do this, Case Manager relies on its database. A query is made to theÂ **AM_EVENT_<channel>** table (where channel is equal to the channel of the event being analyzed), attempting to find another event with the same ID (i.e. the field configured as the **correlation** identifier, within a configured time interval [e.g. update_time_frame_secs]). If such an event exists, then the current event is an update. Otherwise, the current event is new, because it's the first event of a particular transaction that Case Manager is seeing. Recall that due to the possibility of events arriving out of order, the first event that Case Manager sees of a particular transaction will always be considered to be new, regardless of the business semantics.

So, in essence, an event is considered to be an update if there's an event in theÂ **AM_EVENT_<channel>** tableÂ with the same identifier within the time windowÂ **now** minus the configured **update time frame**. Let's see an example, to help visualize this:

1. An event A arrives with timestamp 10000 and **correlationÂ ID** 123.
2. One hour later, an event B arrives with timestamp 13600 andÂ **correlation ID** 123.
3. Since the configured update time frame is **5000**, Case Manager will perform a query to retrieve events withÂ **correlation ID** equal to 123 that have a timestamp greater than or equal toÂ 8600 (**13600** minusÂ **update time frame**).
4. Since event A has a timestamp of 10000 (which is greater than 8600) and the same correlation ID, we will find it, thus concluding that event B is an update.

danger

The importance of the **update_time_frame** property should be clear. If not present, CM will have to look in the entireÂ **AM_EVENT_<channel>** table, which will be a very expensive operation. By having the property configured, we will only need to look into a smaller window. Projects should define and be aware of how far in time an update may arrive, and configure this property accordingly. Failure to configure it will result in CM's event ingestion operating at a significantly degraded rate.

After the current event is labeled as new or an update, their flows are slightly different.

Process the eventâ□□

The type of work done for each event depends heavily on whether the event is new or an update. So, after deciding which it is, CM can follow one of two flows: new event processing or event update processing.

New event processingâ□□

The flow of ingesting new events is significantly simpler than event updates, so we'll look into it first. In this case,Â CM only requires writing the event to the appropriate tables in the database.

First, CM will write the event in theÂ **AM_EVENT_<channel>** table, by looking at the configuredÂ **schema**. Only the fields present in the configured schema will be written into this table. If a certain field contains anÂ **alias** for example, then the column name used will be theÂ name of theÂ **alias**. Besides these fields, Case Manager will also infer and write the initial states for any state machine configured as such.

A similar entry to the one added above is then added to theÂ **AM_EVENT_COMMON** table, except only the schema fields configured as common will be written here. Regarding initial states, only initial states for common state machines will be written.

Before writing in the above tables, CM requires inferring the required initial states, so that it knows what to write. Depending on the configuration of each **state machine** as well as what is configured in the **initial_state** property of each state machine, Case Manager will set the initial state of each one. Each state machine must be configured with an initial state policy (or none, if desired), which will tell Case Manager what to do in this step:

- **PULSE** â□□The initial state is defined by Pulse, i.e. it is present in an event field (**outcomes**). Each state of the state machine then contains a configured **outcome_value**. This value is used to match against the value present in the event payload and thus derive the initial state.
- **DEFAULT** â□□When the state machine's initial state policy is DEFAULT, a **default_state** must also be configured. This is the value that will be considered as the initial state.
- **UNKNOWN** (nothing configured) â□□If no initial state policy is configured for a state machine, nothing will be written on event ingestion.

After deriving the initial state for each of the event's state machines, Case Manager can then perform the insertion of the event data into **AM_EVENT_<channel>** and **AM_EVENT_COMMON**.

There are still a few steps before Case Manager is done processing the event, but they are common with new events and event updates, so we will analyze them after reading about the processing of event updates.

Before moving on, there are a few corner cases to be aware of at this stage. It's possible that Case Manager may receive two or more events belonging to the same transaction (i.e. with the same correlation ID) very close in time, meaning that they may be processed by different threads or even different Case Manager nodes simultaneously. If two (or more) events of the same transaction (e.g. the original and an update) arrive and are processed simultaneously by different nodes, they will both be recognized as **new** events because CM **hasn't seen** that particular transaction before (i.e. it's not present in the database yet, as per step 1 of event ingestion). This means that the different CM nodes/threads will both attempt to write the event to the database as beingÂ **new.** Only one of these attempts will succeed and the other will fail due to a primary key constraint violation, as it's not possible to insert two rows with the same primary key.Â This is because Case Manager uses theÂ **EVENT_ID** column as its primary key, which guarantees that no two concurrent threads/nodes write events with the same ID at the same time.Â This allows CM to be aware of this issue and tackle it when it occurs. Every time an event fails to be written to the database, CM will retry the event ingestion process up to five times. On retry, the failed event will then be recognized as an update (as per step 1 of event ingestion and be written successfully, since the operation executed on the database will no longer be anÂ **insert** but anÂ **update**.

Event update processingâ□□

An update is similar to a new event in the sense that it will writeÂ to theÂ **AM_EVENT_<channel>** and theÂ **AM_EVENT_COMMON** tables, except that it will perform an update to the existing event on these tables, instead of inserting a new row. However, there's a little bit more logic involved in updates. An update may be one of two different types: **normal** orÂ **partial** updates (depending on whetherÂ **partial_updates** are configured). AÂ **normal** update will update everything (i.e. all the fields present in the event update schema), whereas a **partial** update will only update fields that areÂ **not marked** as **ignore_on_update** (i.e.Â **ignore_on_update** set toÂ **true** in the schema configuration). Additionally, aÂ **partial** update will contain a schema field that explicitly states whether the event is the original or a partial update.

Also note that a **normal** update A will only be performedÂ **if** the timestamp of the update **happens after** the timestamp of the event currently on the database. This means that if a more recent update B was processed before update A, then update A will have no effect and will be skipped. This is done in order to guarantee that at any time, the database always reflects the most recent event version, as it would make no sense to overwrite the most recent event data with older information.

In order to better illustrate the difference between **normal** andÂ **partial** updates, consider two events (A and B), belonging to the same transaction, and a CM deployment whereÂ **partial updates are disabled**:

Event A (original):

ID	TIMESTAMP	VERSION_ID	VERSION_TIMESTAMP	FIELD_A	FIELD_B	FIELD_C	EVENT_TYPE
123	1000	1	1000	abc	99	123.2	START

When CM processes this event, it will insert a row intoÂ **AM_EVENT_<channel>** that looks like the following:

ID	TIMESTAMP	VERSION_ID	VERSION_TIMESTAMP	FIELD_A	FIELD_B	FIELD_C	EVENT_TYPE
123	1000	1	1000	abc	99	123.2	START

Event B (update):

ID	TIMESTAMP	VERSION_ID	VERSION_TIMESTAMP	FIELD_A	FIELD_B	FIELD_C	EVENT_TYPE
123	5000	2	5000	abc_updated	100	300.55	UPDATE

When **event B** arrives, CM will update the fields accordingly inÂ **AM_EVENT**_, which will then look like the following:

ID	TIMESTAMP	VERSION_ID	VERSION_TIMESTAMP	FIELD_A	FIELD_B	FIELD_C	EVENT_TYPE
123	5000	2	5000	abc_updated	100	300.55	UPDATE

Now, consider the same example withÂ **partial updates enabled** andÂ **field B and field C** with asÂ **ignore on update set to true.** After **updateÂ B**, the row inÂ **AM_EVENT_<channel>** will look like the following:

ID	TIMESTAMP	VERSION_ID	VERSION_TIMESTAMP	FIELD_A	FIELD_B	FIELD_C	EVENT_TYPE
123	5000	2	5000	abc_updated	99 (not updated because it's set to `ignore_on_update`)	123.2 (not updated because it's set to `ignore_on_update`)	UPDATE

With partial updates enabled, fields B and C weren't updated by the update event, regardless of their value inÂ **event B**, because they were set to be ignored.

Out of order events - Normal Updatesâ

There's another nuance to event updates: events arriving out of order. As an example, consider the following event to beÂ **event A**:

ID	TIMESTAMP	VERSION_ID	VERSION_TIMESTAMP	FIELD_A	FIELD_B	FIELD_C
123	1000	1	1000	abc	99	123.2

And **eventÂ B (an update):**

ID	TIMESTAMP	VERSION_ID	VERSION_TIMESTAMP	FIELD_A	FIELD_B	FIELD_C
123	5000	2	5000	abc_updated	100	300.55

If CM processes **eventÂ B** first due to asynchronicity, then the row of the event in theÂ **AM_EVENT_<channel>** table will be as follows:

ID	TIMESTAMP	VERSION_ID	VERSION_TIMESTAMP	FIELD_A	FIELD_B	FIELD_C
123	5000	2	5000	abc_updated	100	300.55

Which would be exactly the same as if CM processedÂ **event A** and only after,Â **event B**. So, if we processÂ the events out of order, we don't need to do anything when processingÂ **event A.**

During the processing of an update, Case Manager may detect that the current event actually occurred earlier (i.e. happened before from a business perspective) than the version of the event that's currently stored.Â This happens because events may arrive out of order, and CM processes an **event B** before processing the **event A**. First,Â **event B** will be considered as a new event (because CM hasn't seen any event with thatÂ **correlation ID** yet). Then, when processing the **event A** (after persistingÂ **event B**), Case Manager will determine that it is an update because it already saw an event for that transaction before. However, when actually processing the update, CM will verify that the version timestamp (i.e. the timestamp of thisÂ **version** of the transaction) for **event A** is actually lower than the version timestamp from the event that was already processed (**event B**). With this in mind, Case Manager realizes that the event update that it's processing is actually older than the event it has already processed. Since theÂ **AM_EVENT_<channel>** andÂ **AM_EVENT_COMMON** tables already contain the most recent event version (because CM already processed the update), there's nothing to be done.

In summary, CM will always discard events with **older versions** than the one that is currently stored on the database, as it already contains the most recent version. Bear in mind, however, that these events will nonetheless update theÂ **alert** flag on the event, even if it's older than the current version. This is because if an event is an alert, it will stay that way forever. Additionally, in these circumstances, all of the remaining event processing and post-processing tasks will be executed (i.e.Â Explanations,Â persisting to CM's event storage andÂ event post-processing tasks such as triggering automation rules and extensions), the only part that is actually skipped is the updates on the database tables (i.e. this step of event ingestion).

All of this is the case except if partial updates are enabled, as we'll see subsequently.

Out of order events - Partial Updatesâ

If we process out of order events (e.g. process an event update before the original) but have partial updates enabled, Case Manager will have to perform a slightly different logic when processing the original event. Consider the same example as before, where there are two events belonging to the same transaction:Â **event A** (the original) andÂ **event B** (the update). If we process **event B** first, then all fields will be set as if it were a new event, with the exception of fields marked as **ignore_on_update**. Then, when processing **event A** we can't simply ignore it because we already have the most recent version. We need to detect which fields are to be ignored on update and writeÂ **only** those fields. To help visualize this, take the example from before where fieldsÂ **B** andÂ **C** are configured withÂ **ignore_on_update** set toÂ **true.**

After processingÂ **event B** first,Â **AM_EVENT_<channel>** will look like this:

ID	TIMESTAMP	VERSION_ID	VERSION_TIMESTAMP	FIELD_A	FIELD_B	FIELD_C	EVENT_TYPE
123	5000	2	5000	abc_updated			UPDATE

Note that fields **B** and **C** are not written, because they areÂ marked asÂ **ignore_on_update** and Case ManagerÂ knows thatÂ **event B** is a partial update, even though it hasn't seen any event for that transaction before. This is due to the **EVENT_TYPE** column being set toÂ **UPDATE** (e.g. one of the configured partial updates values).

Later, when processingÂ **event A**, because we haveÂ **partial updates enabled**, we will only write the fields that **weren't written** because they are marked asÂ **ignore_on_update**. So, we will only write fieldsÂ **B** andÂ **C** because, regardless of the amount of updates that come in, their value should always be equal to the value that they were at in the original event (in this caseÂ **event A**, which CM knows is the original because its **EVENT_TYPE** column is set to **START**). With these corrections performed, the entry inÂ **AM_EVENT_<channel>** will look like this:

ID	TIMESTAMP	VERSION_ID	VERSION_TIMESTAMP	FIELD_A	FIELD_B	FIELD_C	EVENT_TYPE
123	5000	2	5000	abc_updated	99	123.2	UPDATE

Which, as we saw in the previous example where the events were processed in order, is exactly what should be present in the table after the two events have been processed.

Before moving on, imagine a different scenario, where we have three events of the same transaction: event A (original), event B (update), event C (update). From a business perspective, these events occurred in the following order: event A â□□event B â□□event C. Now imagine Case Manager processes event C first and event B second. Just like what happens in normal updates, event B will be ignored because Case Manager already contains the most recent version (i.e. event C). In both of these cases (processing event C and event B) none of the fields marked asÂ **ignore_on_update** will be written. Eventually, when event A arrives, all of the fields marked as **ignore_on_update** will be written, as we saw before. After the three events arrive, regardless of their order, the database will reflect a state equal to the one it would be in if the events arrived in their correct order.

After step 2 (persisting the event) we move on to step 3: persisting explanations.

Explanationsâ□□

If the event being processed contains any explanations, they will be written in this step. The flow here is quite simple, and doesn't require us going into much detail. Case Manager will examine the event payload, extract the explanations and then add them to CM'sÂ **explanations batch,** which will eventually persist the explanations into theÂ **AM_EVENT_EXPLANATION** table.

Worth noting are this batch's configuration properties:

- `batch_storage_size` â□□The amount of entries after which the explanations batch will perform a flush.
- `batch_storage_timeout` â□□The time after which the explanations batch will perform a flush, regardless of the amount of entries present.
- `max_queued_explanations` â□□The maximum number of queued explanations for persistence by the batch. The explanations present in this queue are processed by the explanations batch. If the queue becomes full, any threads attempting to add more explanations to it will block until space is available.

Writing the event into CM's event storageâ□□

After persisting the event's explanations (if any), we move onto the last step of the main event processing flow: writing the event into CM's event storage (AM_EVENT_STORAGE table). This step is simple and doesn't change regardless of whether the event is new or an update. Every single event that passes through Case Manager will contain an entry in theÂ **AM_EVENT_STORAGE** table, which stores all seen event versions and their payload, inÂ **JSON.**

As is the case with explanations, events are written into Case Manager's event storage in batches (although a different batch is used).

The configuration properties for this batch are similar to what we've seen before:

- `batch_size` â□□The amount of entries after which the event storage batch will perform a flush.
- `batch_timeout` â□□The time after which the event storage batch will perform a flush, regardless of the amount of entries present.
- `queue_size` â□□The size of the buffer that holds events waiting to be added to the batch.

The last property **queue_size** is worth explaning. Essentially, before being added to the batch, the events are placed into a buffer. Then, a single-threaded dedicated worker extracts events from this buffer and places them into this batch. So, the event processing flow ends when Case Manager adds the event to this buffer. Eventually, the batch worker will add it to the batch and it will be persisted to theÂ **AM_EVENT_STORAGE** table. TheÂ **queue_size** property allows the definition of this buffer's size.

There are a few nuances that you should be aware of when configuring this batch's properties. Since writing to theÂ **AM_EVENT_STORAGE** table is an expensive operation (due to writing the entire event payload inÂ JSON), problems may arise if the batch size is too high. If the batch contains a significant amount of entries, the eventual flush that occurs may take a significant amount of time. If the time it takes to flush the then CM's event ingestion will slow down significantly because threads (CM workers processing events) attempting to add events to the buffer for the eventual addition to the batch will block due to the buffer being full. In this case, you should consider either increasing the buffer size or decreasing the batch size, so that these two work in balance.

Case Manager uses a special batch implementation which performs an action after every batch flush for each of the events that were just persisted. This is the step that triggers the final step in Case Manager's event ingestion: Event post-processing.

Event post-processingâ□□

As mentioned in the previous step, after the event storage batch performs a flush, each of the events will be sent to this final step: post-processing, where final actions are taken on each of the events. There are several post-processing actions and they depend on the type of event, so we will look into each of them in more detail.

New event post-processingâ□□

The post-processing operations for a new event are, similarly to step 2, simpler than the post-processing actions for event updates. The post-processing actions performed for new events are the following:

1. Notify external systems of the event, if necessary. â□□If configured, Case Manager sends information to external systems (e.g. Insights, elasticsearch, etc.) about new events. So, after an event was processed, this step ensures that its data is sent to these systems.
2. Notify external systems of the event's initial states, if necessary. â□□As before, if configured, Case Manager will send information regarding its initial states to external systems. This is similar to step 1, except it only sends information about the event's states, rather than the event itself.
3. Trigger custom extensionsÂ â□□This step performs the triggering of custom extensions. If a project has any extensions that trigger on event arrival, this is the step in which they will be executed. Bear in mind that the extension will be executed by the same thread that is handling the event post-processing, so heavy computations should be handled with care. If the event post-processing step starts to slow down in relation to the main event processing flow, CM's throughput will eventually fall because the main CM workers will be blocked by the lack of resources to execute event post-processing.
4. Trigger automationsÂ â□□Finally, automations for the new event will be triggered. That is any automation with the triggers: **Event Arrives and Most Recent Event Version received**. Automation rules will be executed by a thread belonging to the **automation workers executor**. This is a limited resource, so they should be used sparingly as they may cause CM's throughput to plummet.

After these steps are completed, then the new event has officially reached the end of its journey.

Event update post-processingâ□□

The post-processing steps of event updates are similar to the steps for new events, but with added complexity:

1. Notify external systems of the event, if necessary. â□□If configured, Case Manager sends information to external systems (e.g. Insights, elasticsearch, etc.) about event updates. So, after an event update was processed, this step ensures that its data is sent to these systems, so that they can update their version of the event accordingly.
2. Trigger custom extensionsÂ â□□This step performs the triggering of custom extensions. If a project has any extensions that trigger on event update, this is the step in which they will be executed. Bear in mind that the extension will be executed by the same thread that is handling the event post-processing, so heavy computations should be handled with care. If the event post-processing step starts to slow down in relation to the main event processing flow, CM's throughput will eventually fall because the main CM workers will be blocked by the lack of resources to execute event post-processing.
3. Trigger automationsÂ â□□Finally, automations for the new event will be triggered. That is any automation with the triggers:Â **Event Updated**Â andÂ **Most Recent Event Version received** (if the event update was indeed the most recent version). Automation rules will be executed by a thread belonging to the **automation workers executor**. This is a limited resource, so they should be used sparingly as they may cause CM's throughput to plummet.

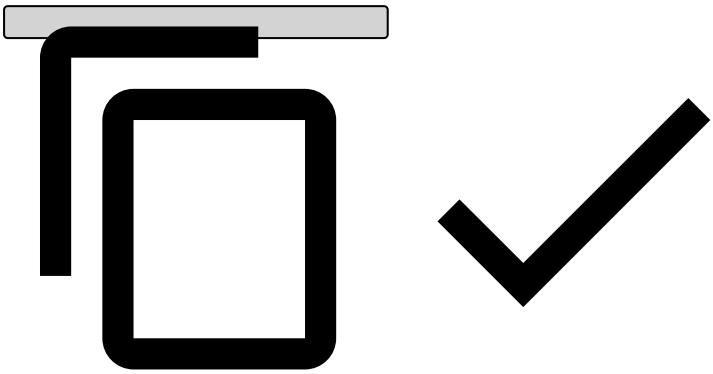
The post-processing step that differs mostly from new events is the triggering of automations, because of a small detail. Automations with theÂ **Event Updated** trigger require information not only about the current event, but about the original event (the one that was updated). In order to fetch it for the automation rule's usage, we have to look into Case Manager's event storage. The problem is that the event may not be present due a few reasons:

- The event is present in the event storage batch, but it wasn't flushed yet.
- The event is being processed by a different physical Case Manager node

If one of these two scenarios occurs, then we won't be able to look into CM's event storage to fetch the original event in order to trigger the automation rule, merely because it isn't present yet. This circumstance should only occur if both a new and an event update arrive very close in time. In any case, Case Manager handles this with a retry mechanism which re-queues the automation triggering post-processing action for a later re-attempt. The failed automation triggering action will

be attempted up to five timesÂ **AND** a predetermined timeout has been reached. The reason why both these conditions have to be achieved is for the scenario where Case Manager's event ingestion is quiet and the automation trigger re-attempts will be processed almost instantly, giving almost no time for the event we're looking for to actually be persisted to the database. The predetermined timeout can be calculated in the following way:

```
retry_timeout = 2 x batch_timeout + SALT
```



That is, we are allowing for two batch flushes (plus a fixed "SALT" value: *250ms*) to occur before we give up attempting to look for the original event in order to trigger the automation. If this amount of time passes (as well as five retries) and Case Manager is still not able to find the original event in the event storage, then it will give up triggering the automation and it will be logged to the automations dead-letter.

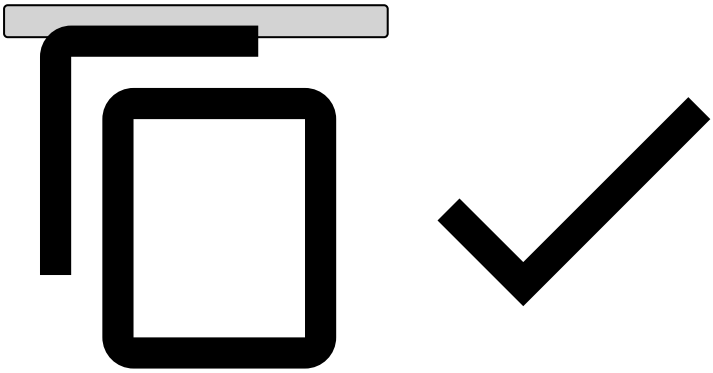
After these steps are complete, then the event update has officially reached the end of its journey.

Disabling event updatesâ□□

It's possible that the business semantics of a particular Case Manager deployment do not require processing event updates. This allows for a great performance improvement in event ingestion by skipping the first step in event ingestion (verifying whether each event is an update) and, more importantly, by batching the entire event ingestion process.

This can be achieved by enabling the following property:

```
ignore_event_updates: true
```



When this is done, Case Manager will use a completely different logic to process event ingestion. The basic premise is the same as described above, with a few important changes:

- No verification is made to determine whether each event is an update (which avoids unnecessary DB calls).
- All writes to AM_EVENT_<channel>, AM_EVENT_COMMON and AM_EVENT_STORAGE are batched.

Essentially, the event's journey is as follows:

1. Add event to EventCommonBatch
2. Add event to EventBatch
    1. When this batch flushes, execute the following for the n flushed events (EventBatch post-processing):
        1. Apply transformations to flushed events
        2. Trigger initial state triggers (automations/extensions)

3. Persist explanations
4. Add events to EventStorageBatch
    1. When this batch flushes, execute post-processing logic (EventStorageBatch new event post-processing, described above).

Note that this approach means that the event can fail in different, independent steps. For example, it's possible that the addition to the AM_EVENT_COMMON table succeeds while all of the other steps fail. It's also possible that the post-processing actions of some batch fails, resulting in automations not being triggered, external systems not being notified, explanations not being persisted, and so on.

It's important to be aware of this when re-ingesting events that were sent to the dead-letter, and verify whether they were already persisted to different parts of the database.

Here are the different failure points and their consequences:

Point of Failure	**Result**	**Failure Handler**	**Recovery**
EventCommonBatch flush	Event is not persisted to AM_EVENT_COMMON	Write event to **CM_EVENT_COMMON_DEADLETTER**	Manual recovery (inject events into the AM_EVENT_COMMON table)
EventBatch flush	Event is not persisted to AM_EVENT_<channel>, post-processing actions aren't executed.	Write event to dead-letter	If the event fails here, recovering might be different depending on a few factors: • If the event was correctly added to AM_EVENT_COMMON via the EventCommonBatch, then we can't simply re-inject the event using the admin tool, since it will fail with a duplicate PK violation when inserting again into AM_EVENT_COMMON. In this case recovery should be: 1. Delete event from AM_EVENT_COMMON 2. Re-inject using admin tool; • Otherwise, if the event also failed to be added to AM_EVENT_COMMON (event failed completely, wasn't persisted anywhere), then it's a matter of using the admin tool to re-inject the event.
EventBatch post-processing (flush callback)	Depends on where it fails, but possibly: • Initial state automations not triggered • Event explanations not persisted • Event not added to EventStorageBatch â□□ Event not persisted to AM_EVENT_STORAGE	Write event to dead-letter	Manual recovery (re-inject events), however it will require a few validations/ assumptions: • Event may be present in AM_EVENT_STORAGE, if so it must be deleted before re-injection. • Event may be present in AM_EVENT_COMMON, if so it must be deleted before re-injection. • Explanations may already be present, if so they must be deleted before re-injection. • Initial state automations may have already been triggered, and their side effects may already be observed. In this case, they will be re-triggered.

Point of Failure	Result	Failure Handler	Recovery
EventStorageBatch flush	Event is not persisted to AM_EVENT_STORAGE, post-processing actions are not executed	Write event to event storage dead-letter	Manual recovery (inject events into the AM_EVENT_STORAGE table)
EventStorageBatch post-processing (flush callback)	Depends on where it fails, but possibly: • Event not sent to external systems • On event arrival extensions not triggered • On event arrival automations not triggered	None	N/A

The point of failure can be determined by analyzing the Case Manager logs.

Case Manager's Thread Pools for Ingestionâ□□

Case Manager has a few important thread pools that deal with event ingestion:

- **CM Workers:** responsible for ingesting events and executing the event processing steps previously described (step 1 to 3).
- **CM Automation Workers:** responsible for processing automation rules (both condition evaluation and action execution).
- **Batch Flush Workers (one per batch):** responsible for executing the post-flush callback actions for some batches, such as the event storage batch (to persist events to AM_EVENT_STORAGE), the automation action batches (post-processing actions related to automation rules) and the event batch (when **ignore_event_updates** is **true**). Note that there is one instance of **Batch Flush Workers** for each of the above batches, so they work independently of one another.

These thread pools are configurable in terms of their size (i.e. amount of workers), but the Automation Workers feature a different important property: **max_queued_automations**.

This property limits the amount of "actions" or "work items" that may be sent to the thread pool. If, for some reason, it becomes full, then other entities attempting to add more work to the Automation Workers thread pool will block until space is available. This is important to be aware of because one of the entities that adds work to the Automation Workers is the event storage batch (which, as we saw previously, triggers post-processing actions after each flush). If, for some reason, the work queue for the Automation Workers is full, then the batch will take a longer time to execute post-processing actions, which will slow the batch flush down. Eventually, the batch buffer will fill and calling threads (i.e. workers from the Batch Flush thread pool) will block waiting for space to become available. This means that if the CM Automation Workers is slow, eventually the Batch Flush Workers will be slow. This will result in batch flushes taking longer, causing CM's throughput to be affected.

On the opposite side, configuring too high of a value may result in memory problems, so a balance needs to be achieved.

A similar property exists for the **Batch Flush Workers** executor: **batch_flush_queue_size**. This property controls the maximum amount of batch flush post-processing actions queued for execution. As is the case with the **max_queued_automations** property, if the work queue reaches the configured size, then further threads attempting to add work to these threads will block. This means that the batches will be unable to perform the batch flush post-processing actions and will eventually stop flushing the batches, essentially stalling event ingestion and/or automation rule action execution.